Seneca Polytechnic
AIG230 — Natural Language Processing
Postgraduate Certificate in Artificial Intelligence

Transformer-Based Neural Machine Translation

A single bidirectional English ↔ Spanish model

Group 7

Jose Luis Sanchez Noriega
Bikash Subedi

Instructor: Ellie Azizi

Final Project — Option 1
Corpus: the Tatoeba Project (English–Spanish)
Framework: PyTorch

August 11, 2026

Abstract

We build a complete neural machine translation system for English and Spanish using a transformer encoder–decoder implemented from primitives in PyTorch. The distinguishing design choice is that *one* set of weights serves *both* translation directions: source and target draw on a single joint subword vocabulary, the embedding matrix is shared three ways between the encoder, the decoder and the output projection, and a reserved direction tag prepended to the source selects the output language. This halves the parameter budget relative to two separate models while doubling the supervision each parameter receives.

The corpus is reconstructed from the official Tatoeba exports by joining 2,033,133 English sentences and 441,233 Spanish sentences through 28,361,472 translation links, yielding 282,902 aligned pairs. We show that the standard practice of splitting such a corpus uniformly at random leaks between train and test — Tatoeba is a translation *graph*, not a list of independent pairs — and partition connected components instead, which removes the leak entirely.

We report BLEU and chrF2 in both directions against two baselines: a capacity-matched recurrent sequence-to-sequence model with Bahdanau attention, and an ablation isolating the effect of initialising embeddings from MUSE cross-lingual vectors. An automated error analysis classifies every test translation into named failure modes and surfaces the twenty worst cases for manual inspection. The trained model is served through a Gradio application that translates either direction interactively and displays the cross-attention alignment behind each output.

> **Reproducibility.** Every number, table and figure in this report is generated from JSON artefacts written by the pipeline (`reports/build_report_data.py` and `nmt.viz.make_figures`). Nothing is transcribed by hand, so the document cannot drift out of step with the code and checkpoints in the repository.

Contents

1 Introduction

1.1 The task

Machine translation maps a sentence in a source language to a sentence in a target language that preserves its meaning. It is a *sequence-to-sequence* problem with three properties that make it hard and that shape every decision in this project:

- **The output length is not determined by the input length.** "I'm tired" is two words in English and two in Spanish ("Estoy cansado"), but "I'll see you tomorrow" is five and "Te veré mañana" is three. The model must decide when to stop.
- **The alignment is not monotonic.** English places adjectives before nouns and Spanish generally after: "the red house" becomes "la casa roja". A model that could only attend to source positions in order would be unable to produce this.
- **The target is not unique.** "I'm tired" is equally well "Estoy cansado", "Estoy cansada" or "Tengo sueño". Any loss that demands probability 1 on one particular reference is asking for something false — a fact that directly motivates the label smoothing of Section 5.

1.2 Why one model for both directions

The obvious design is two models, one per direction. We chose a single bidirectional model instead, following the multilingual tagging approach of Johnson et al. [2]. Two reserved tokens, `<2en>` and `<2es>`, are prepended to the source sentence:

```
<2es> I am tired .      →   Estoy cansado .
<2en> Estoy cansado .   →   I am tired .
```

Every cleaned sentence pair therefore produces *two* training examples, one per direction. Three consequences follow, and they are the reason for the choice:

1. **Twice the supervision per parameter.** The same encoder must represent both English and Spanish sentences, and the same decoder must generate both. On a corpus of this size — around a quarter of a million pairs, two orders of magnitude smaller than the WMT datasets transformers were designed for — data, not compute, is the binding constraint, and doubling the effective corpus is worth more than the capacity given up.
2. **A shared vocabulary becomes possible, and then necessary.** Since the decoder emits into the same space the encoder reads from, source and target must share one inventory. This in turn permits three-way weight tying (Section 4), removing roughly a third of the parameters.
3. **Cognates share statistical strength.** "nation"/"nación", "important"/"importante" and "possible"/"posible" share subword units under a joint BPE model, so evidence about one improves the representation of the other.

The cost is capacity: one model of a given size must hold both directions. Section 6 reports what that costs in BLEU.

1.3 Contributions

1. A transformer encoder–decoder written from primitives — attention, positional encoding, masking, residual blocks — rather than assembled from `torch.nn.Transformer`, with the training loop written by hand as the brief requires.

2. A corpus construction pipeline that rebuilds the bitext from the official Tatoeba exports and demonstrates, with a measurement, that component-aware splitting is necessary to avoid leakage.
3. A controlled three-way comparison isolating the effect of pre-trained cross-lingual embeddings from the effect of the tokenisation change that normally confounds it.
4. A recurrent baseline matched on data, vocabulary, optimiser and parameter count, so the architectural comparison means something.
5. An automated error analysis that names and counts failure modes across the whole test set rather than reporting a single aggregate score.

2 Data collection and exploration

2.1 Building the corpus from the official exports

The Tatoeba Project publishes per-language sentence dumps and a single global table of translation links; it does not publish a ready-made English–Spanish bitext. We reconstruct one, which keeps the corpus pinned to a dated snapshot we control and preserves the Tatoeba sentence identifiers so that any example quoted in this report can be traced back to the public database.

Three files are downloaded (about 172 MB compressed) and joined in three passes so that peak memory stays proportional to the two languages of interest rather than to the size of the link table:

1. load `eng_sentences.tsv` into `{id: text}` — 2,033,133 rows;
2. load `spa_sentences.tsv` — 441,233 rows;
3. stream `links.csv` (28,361,472 rows), emitting a pair whenever the left identifier is English *and* the right one is Spanish.

Because Tatoeba stores every link in both orientations, that asymmetric test yields each English–Spanish pair exactly once. The join produces 282,902 aligned pairs.

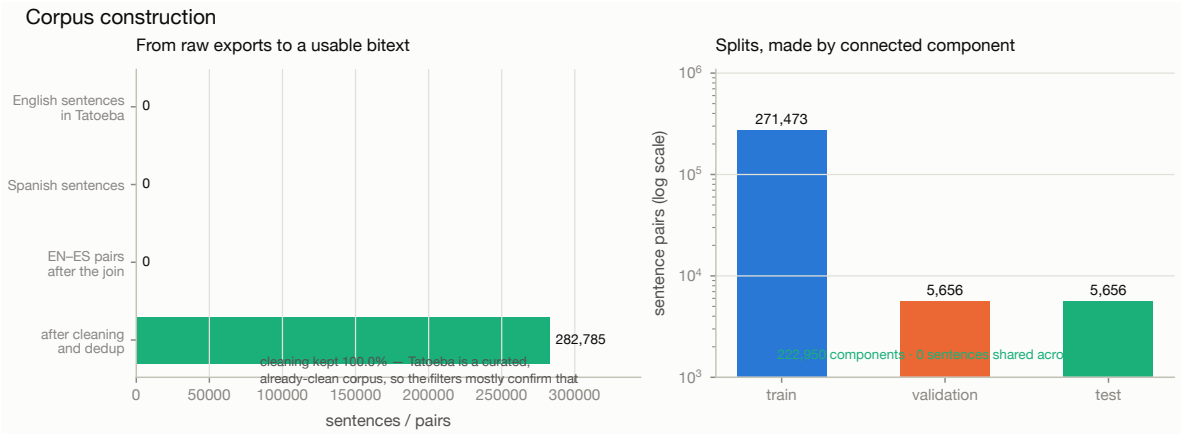

Figure 1: From the raw Tatoeba exports to the final splits. Cleaning retains 99.96% of pairs: Tatoeba is a curated, human-checked corpus, so the filters largely confirm its quality rather than repair it. The splits are made by connected component, and the leakage check is run on every build.

2.2 What the corpus looks like

Three characteristics shape the modelling decisions that follow.

Table 1: Corpus statistics after cleaning. Subword counts use the joint 16,000-entry BPE model fitted on the training split only.

		unique sentences		mean words		mean subwords	
Split	Pairs	EN	ES	EN	ES	EN	ES
Train	271,473	232,777	250,057	6.8	6.6	9.1	8.8
Validation	5,656	5,026	5,330	6.9	6.7	9.4	9.1
Test	5,656	5,032	5,326	6.9	6.7	9.3	9.0

The sentences are short. Both sides average under seven whitespace tokens, and the 95th percentile is around thirteen. Tatoeba is a corpus of everyday example sentences contributed by language learners, not of newswire or parliamentary proceedings. This is convenient — attention is quadratic in sequence length, so short sentences make the model cheap to train — but it also bounds what the system can be expected to do: it will be weakest on exactly the long, syntactically complex sentences that are rarest in training.

Spanish has far more surface forms than English. The training split contains 35,066 distinct English word types against 56,519 Spanish ones, from a near-identical number of running tokens (2,161,611 and 2,145,680). That is a factor of 1.61. The cause is morphology: Spanish marks person, number, tense and mood on the verb (*hablo, hablas, habla, hablamos, habláis, hablan*, and that is one tense of one verb), agrees adjectives and articles for gender and number, and attaches clitic pronouns directly to infinitives and gerunds (*dármelo*). English does almost none of this.

The tail is heavy. 39.6% of English types and 42.5% of Spanish types occur exactly once in training. A word seen once cannot train a useful embedding.

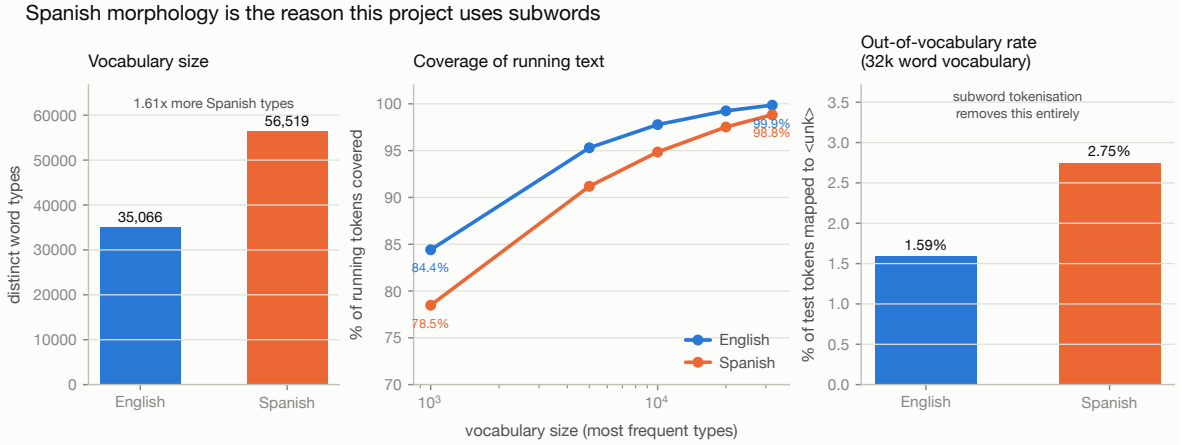

Figure 2: The case for subword tokenisation. Spanish contributes 1.61× as many word types as English (left); a 32,000-entry word vocabulary still leaves 2.75% of Spanish test tokens unknown against 1.59% of English ones (right). Subword tokenisation removes the unknown-word problem entirely, at the cost of longer sequences.

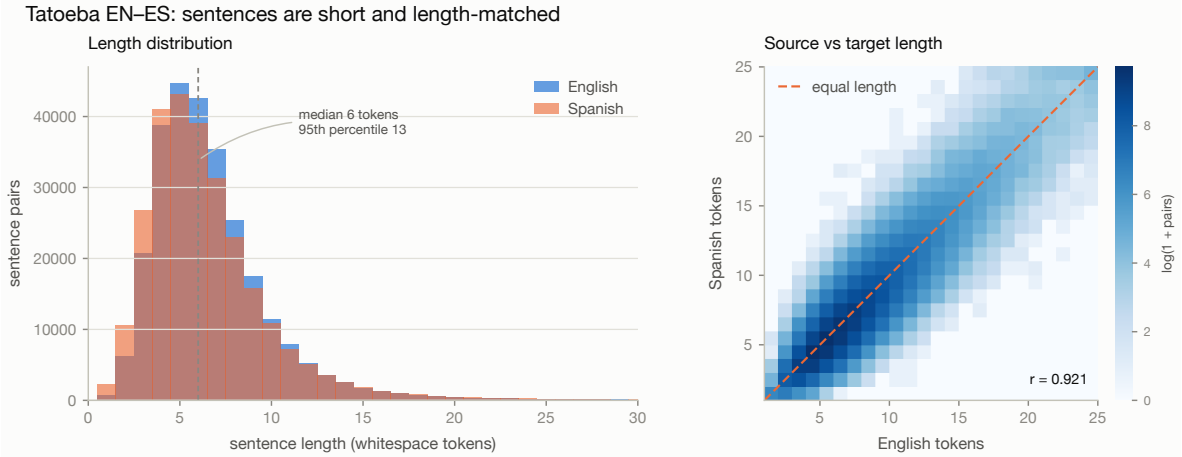

Figure 3: Sentence-length distributions and the relationship between the two sides. The strong correlation is what makes a length-ratio filter a sound way to detect misaligned links.

2.3 Characteristics that will influence the task

Implications carried into the rest of the project.

- *Short sentences* ⇒ a maximum length of 64 tokens discards a negligible tail; batches can be capped by token count to keep memory flat.
- *Spanish morphology* ⇒ a subword vocabulary, and chrF2 reported alongside BLEU because a translation can get a stem right and its inflection wrong, which BLEU scores as a total miss.
- *Heavy tail* ⇒ a modest 16,000-entry vocabulary, so that even the rare half of it is updated often enough to learn.
- *Many-to-many links* ⇒ component-aware splitting (Section 3.3).

3 Preprocessing

Every rule below is a modelling decision, so each is stated with its justification. The guiding principle is *remove noise, preserve meaning*: normalise away distinctions the model should not spend capacity on, and leave intact everything a reader would consider part of the sentence.

3.1 Normalisation

Applied in this order, because the order matters:

1. **Unicode NFC composition.** Spanish accents arrive both pre-composed (ñ = U+00F1) and decomposed (n + U+0303). They render identically but are different strings; without NFC the tokeniser treats *español* as two distinct types.
2. **Control-character removal**, before whitespace collapsing, so that a stripped control character cannot weld two words together.
3. **Typographic folding.** Five kinds of apostrophe, six of quote and seven of dash are mapped to one ASCII form each. Tatoeba is crowd-sourced, so the same sentence appears with curly or straight quotes depending on the contributor's keyboard.
4. **Whitespace collapsing**, last, since the previous steps can introduce runs of spaces.

> **Casing and punctuation are preserved.** Lowercasing shrinks the vocabulary and typically buys a point of BLEU on a small corpus, but it makes the system unable to produce correctly-cased output, which would then need a separate truecasing model at inference. Since the deliverable is an interactive application whose output a human reads, correct casing is part of the product. Likewise, Spanish inverted marks (¿, ¡) are grammatical signal, and translating punctuation is part of translating.

3.2 Filtering

Pairs are discarded when either side is empty after normalisation, contains no alphabetic content ("1997."), contains a URL or e-mail address, contains characters from an unexpected script (mislabelled rows), falls outside 1–64 tokens or 2–400 characters, is identical on both sides, or has a length ratio above 3.0 once both sides reach four tokens. The ratio test is suppressed on very short sentences because "Yes." against "Sí, lo haré." is a legitimate 1:3 pair.

Exact duplicate pairs are removed. Near-duplicates are deliberately *not* removed: Tatoeba's alternative translations of the same sentence are genuine paraphrases and discarding them would throw away real supervision. What must not happen is those paraphrases straddling the train/test boundary — and that is handled by the split, not by the filter.

Cleaning retains 282,785 of 282,902 pairs (99.96%). The high retention is itself a finding worth stating: Tatoeba is human-curated, so the aggressive cleaning pipelines that web-crawled corpora require are not needed here. The filters earn their place by *confirming* data quality and by catching the small number of genuinely broken rows.

3.3 Splitting without leakage

> Tatoeba is a translation *graph*, not a list of independent pairs. One English sentence typically links to several Spanish sentences, and each of those may link to further English sentences. Splitting the pair list uniformly at random therefore leaks: "I'm tired." / "Estoy cansado." lands in training while "I'm tired." / "Estoy cansada." lands in test, and the reported BLEU measures memorisation rather than translation.

The fix is to split *connected components*. Treat every distinct sentence as a node and every link as an edge, merge them with a union–find structure, and assign whole components to a split. Sentences are namespaced by language so that an English string equal to a Spanish string (proper nouns, "No.") does not spuriously merge two unrelated components.

This yields 222,950 components over 282,785 pairs, the largest containing 169 pairs. Components contributing more than four pairs are forced into training: a forty-pair component landing in test would fill the evaluation set with paraphrases of one sentence and make BLEU hostage to whether the model happened to know it.

A leakage assertion runs on every build and is recorded in the split report; it counts sentence strings shared between splits on either side. The current value is **0**.

3.4 Tokenisation

Two tokenisers are fitted — on the *training split only*, since fitting on the full corpus would leak test-set surface forms into the vocabulary and flatter the reported out-of-vocabulary rate.

Joint subword (primary). A SentencePiece BPE model with 16,000 entries, fitted on the concatenation of English and Spanish training text. Sharing one vocabulary is what makes the bidirectional

model possible. Being open-vocabulary, it never emits <unk>. Fertility is 1.35 subword tokens per English word and 1.35 per Spanish word — the price paid for eliminating unknown words.

Word level (for the embedding experiment). A frequency-truncated vocabulary of 32,000 entries with singletons dropped. It exists because pre-trained word vectors are distributed as word-level tables: to initialise embeddings from MUSE, the units the model embeds must *be* words. The cost is a closed vocabulary, leaving 1.59% of English and 2.75% of Spanish test tokens unknown.

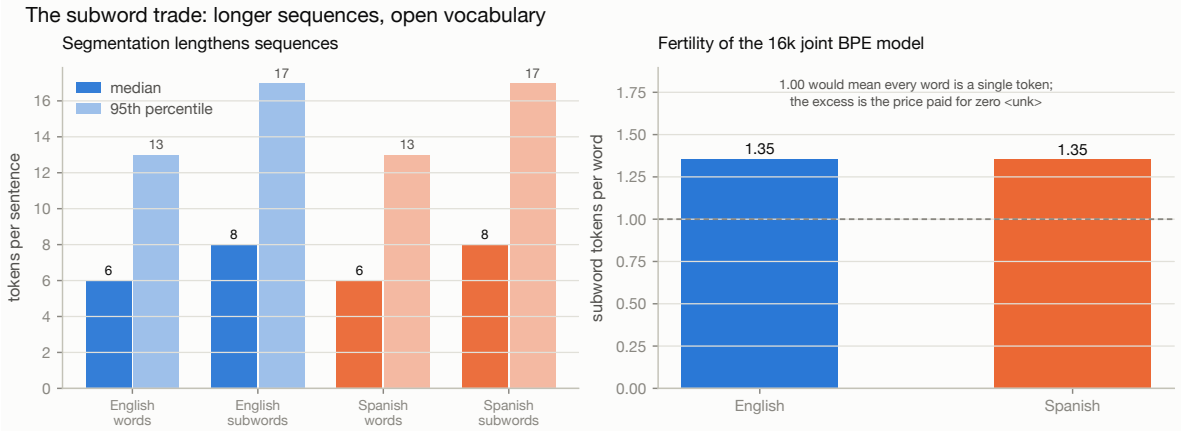

Figure 4: What segmentation costs. Subword tokenisation lengthens sequences by roughly a third, which is real compute, in exchange for never encountering an unknown word.

3.5 Batching

Sequences are padded to a rectangle within each batch, and attention masks mark which positions are real. Batches are formed by *token count* rather than sentence count: the length distribution is right-skewed, so a fixed sentence count would produce batches whose padding waste swings widely depending on whether a long sentence happened to be drawn. The sampler shuffles, takes pools of fifty batches' worth of examples, sorts each pool by length, and emits batches capped at 8,192 tokens. Batch order is then shuffled again so the model does not systematically see short sentences first within an epoch.

4 Model architecture

4.1 Attention

The single operation the architecture is built around is scaled dot-product attention:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{1}$$

Read it as a *differentiable dictionary lookup*. Each query vector is compared against every key by dot product; the scores become a probability distribution through softmax; the output is that distribution's weighted average of the value vectors. A hard lookup would take the single best-matching key — attention takes a soft blend, which is what makes it differentiable and therefore trainable end to end.

Why divide by $\sqrt{d_k}$. If the components of q and k are independent with mean 0 and variance 1, then $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k, so its standard deviation grows as $\sqrt{d_k}$. With

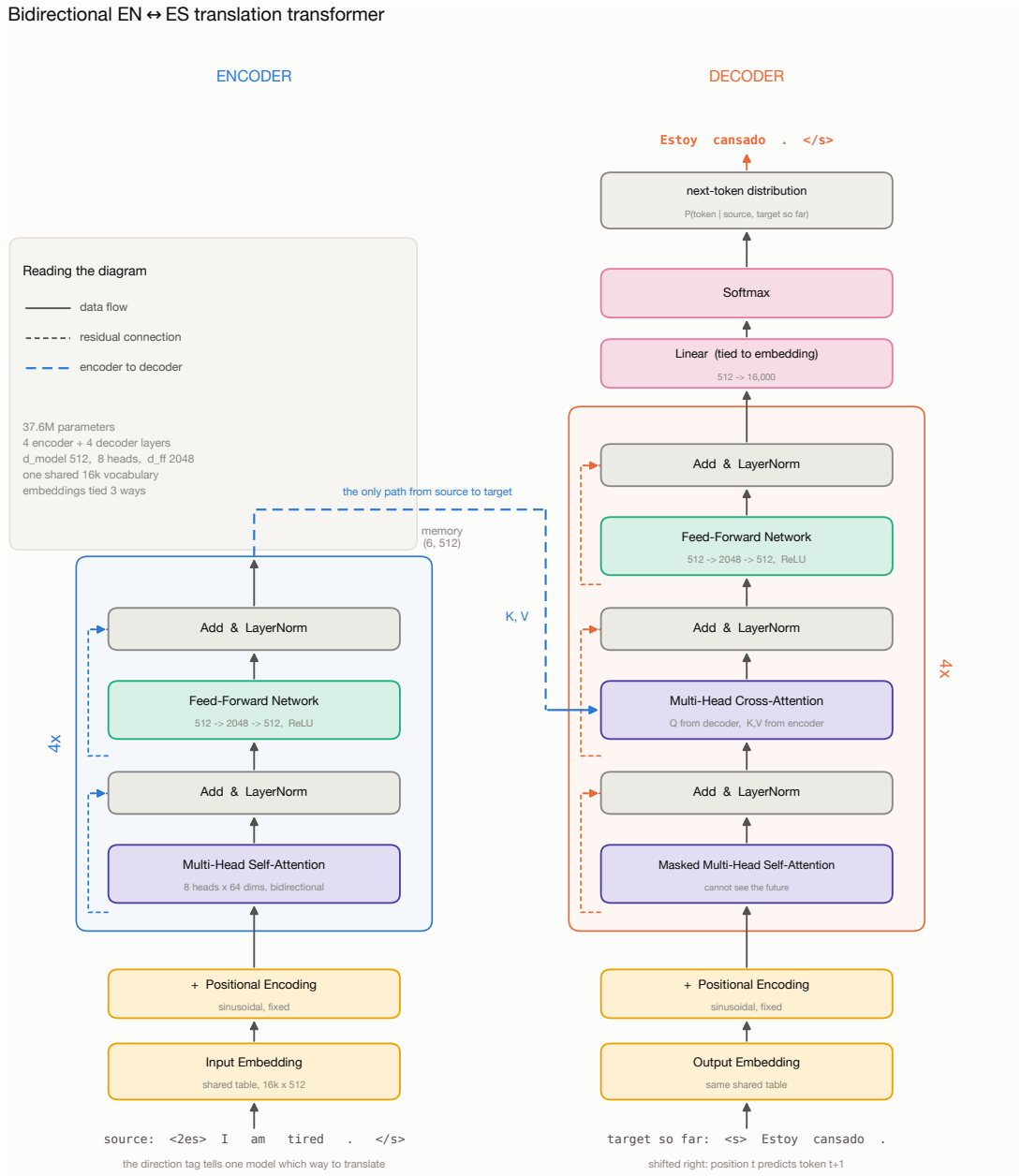

Figure 5: The complete model, annotated with the shapes this project actually uses. Read bottom to top on each side. The encoder turns the tagged source into one vector per source position; the decoder generates the target one token at a time, attending to what it has already produced and, through cross-attention, to the encoder's output.

$d_k = 64$ the raw scores would routinely reach ± 8. Softmax over values that spread out saturates: it becomes nearly one-hot, and the gradient it passes back is proportional to $p(1-p) \approx 0$. Dividing by $\sqrt{d_k}$ restores unit variance and is the difference between a model that trains and one that stalls.

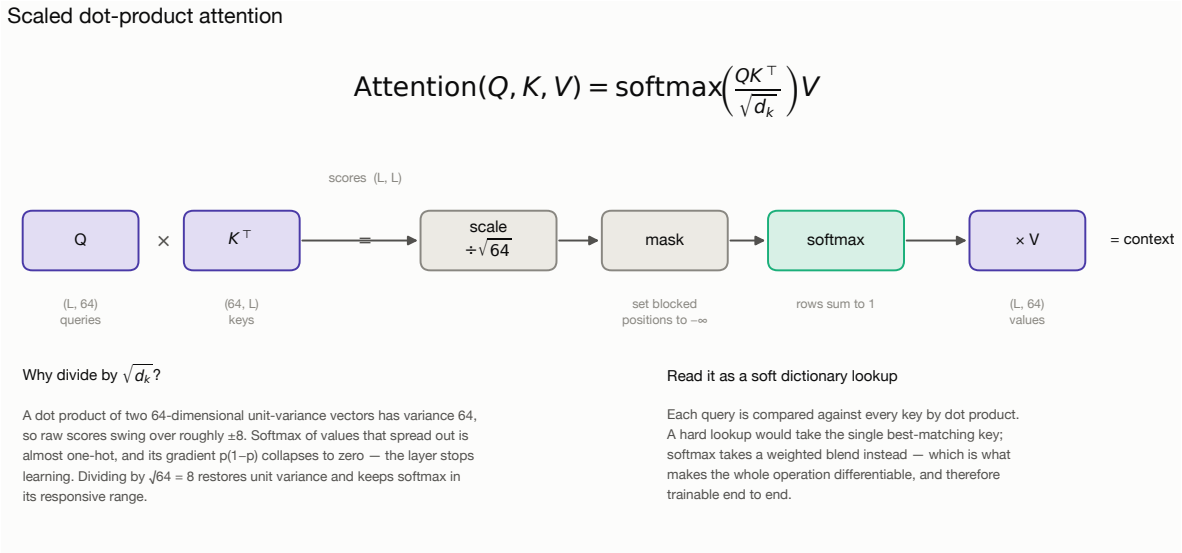

Figure 6: Scaled dot-product attention as a chain of matrix operations, with the shapes used here.

Why multiple heads. A single softmax produces one distribution per query, so one head can attend to essentially one place at a time. Translating "the red house" into "la casa roja" requires simultaneously tracking the noun (for gender agreement on both the article and the adjective) and the adjective (to move it after the noun). Splitting d_{model} into h independent subspaces of size d_{model}/h lets h such relations be attended to in parallel at identical total cost — the split is a reshape, not extra computation.

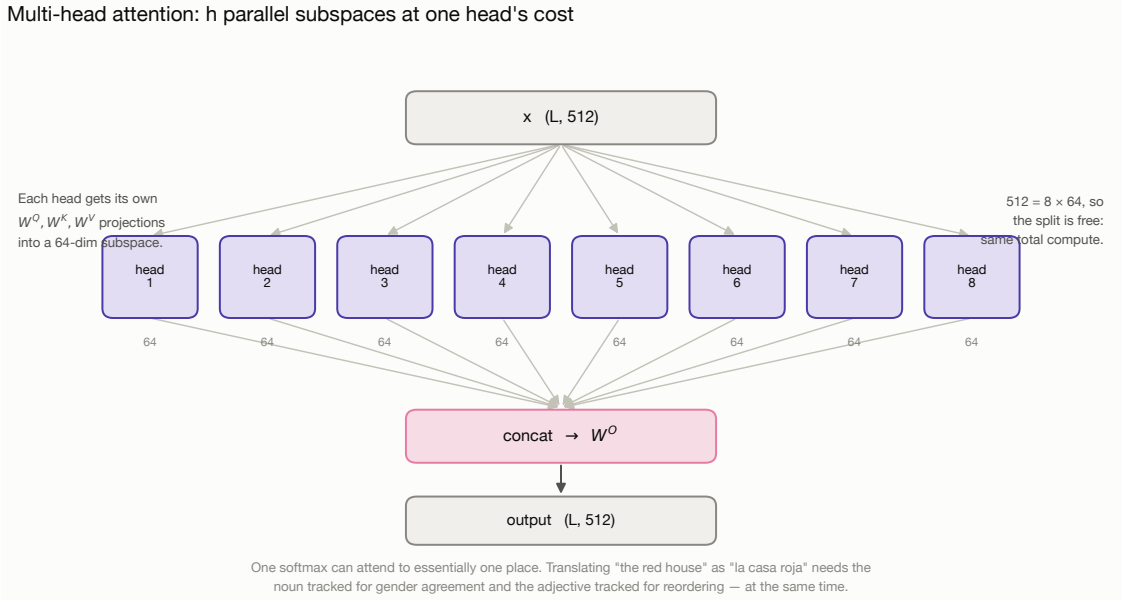

Figure 7: Multi-head attention: h parallel subspaces at one head's cost.

4.2 Positional encoding

Self-attention is permutation-equivariant: it computes a weighted sum, and a sum does not care about order. Shuffle the input words and every attention output is shuffled identically — the model literally cannot distinguish "the dog bit the man" from "the man bit the dog". Position must be injected explicitly. We use fixed sinusoidal encodings,

$$PE_{(pos,\,2i)} = \sin\!\left(pos/10000^{2i/d_{\text{model}}}\right), \tag{2}$$

$$PE_{(pos,\,2i+1)} = \cos\!\left(pos/10000^{2i/d_{\text{model}}}\right), \tag{3}$$

whose motivating property is that *relative* position is a linear function of absolute position: for any fixed offset k there is a matrix M_k — a 2×2 rotation applied per frequency pair — with $PE_{pos+k} = M_k\, PE_{pos}$. Attention can therefore learn "attend three tokens back" as one linear map that works at every position, and because the encoding is a function rather than a lookup table it extends to lengths never seen in training. A learned alternative is implemented for the ablation.

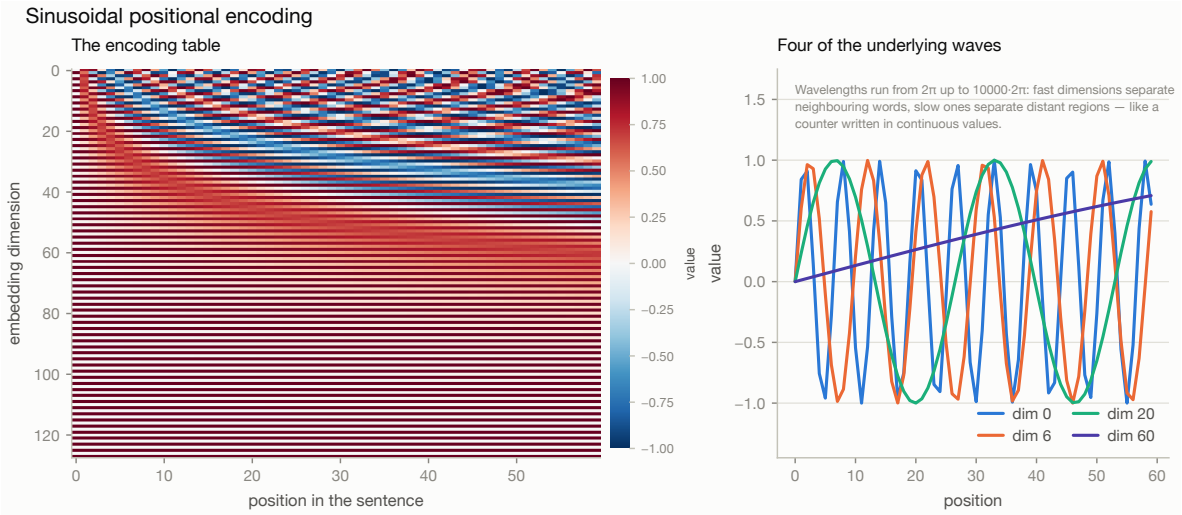

Figure 8: The sinusoidal table and the waves that generate it. Wavelengths run from 2π to $10000 \cdot 2\pi$: fast dimensions separate neighbouring words, slow ones separate distant regions.

4.3 Masking

Two logically different masks are needed, and conflating them is a classic source of silent bugs. The *padding* mask hides filler positions added to square off the batch; without it, the encoder's representation of a short sentence would depend on how long the other sentences in its batch happened to be. The *causal* mask stops the decoder from seeing the future: during training the whole target is supplied at once so all positions compute in parallel, but position t must attend only to positions $\leq t$. Without it the model trivially copies the next token from its input, scores near-perfect training loss, and is useless at inference.

Throughout the codebase `True` means "attend here". The alternative convention is equally common, and mixing the two silently inverts the model.

4.4 Residual connections and layer normalisation

Each sublayer is wrapped in a residual connection around a normalised transformation. The residual gives the gradient a path back to the input that does not pass through the sublayer: differentiating

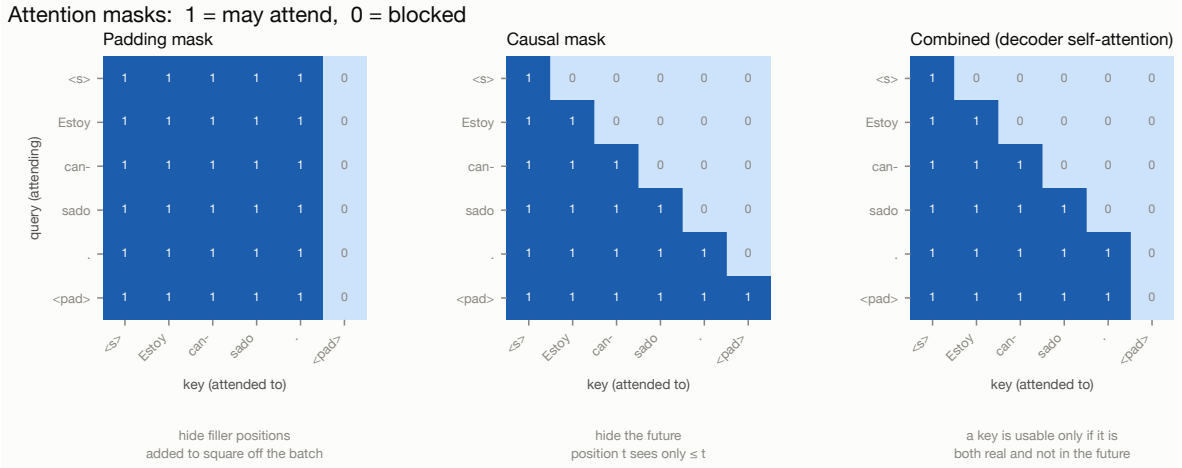

Figure 9: The two masks and their combination for decoder self-attention. A key is usable only if it is both real and not in the future.

$x + f(x)$ gives $1 + f'(x)$, so even if f' vanishes the 1 survives. It also changes what each block must learn — only a *correction* to a running representation, rather than the whole representation.

We use **pre-layer-normalisation**, $x \leftarrow x + \text{Sublayer}(\text{LayerNorm}(x))$, rather than the original paper's post-LN. In pre-LN the residual path runs from input to output un-normalised, so gradients reach the early layers undamped and training is stable at higher learning rates with a shorter warmup. Post-LN needs a carefully tuned warmup to avoid diverging in the first few hundred steps. The practical argument is decisive for a project trained in free Colab sessions: a run that diverges at step 300 is an hour wasted. Pre-LN requires one extra `LayerNorm` at the end of each stack, since the last sublayer's output would otherwise never be normalised.

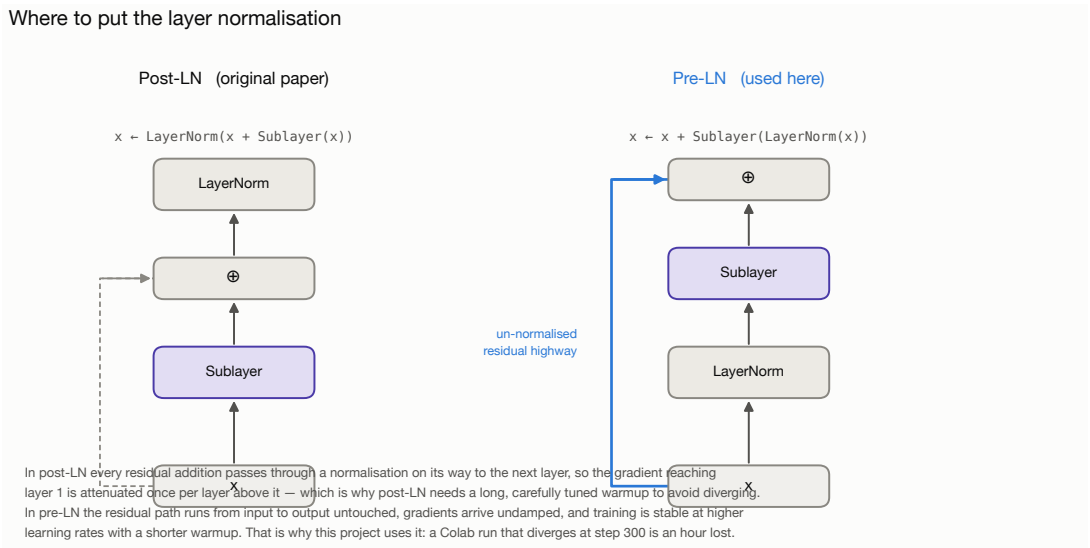

Figure 10: Where to put the layer normalisation, and why it changes training stability.

Table 2: The systems compared. All four share the same data, direction tags, optimiser, decoding code and evaluation.

System	Vocab	Emb. dim	Geometry	Params
no trained runs found				

4.5 Hyper-parameter choices

$d_{\text{model}} = 512$**,** $h = 8$**,** $d_{\text{ff}} = 2048$**.** These are the proportions of "Transformer base" [1]: $d_{\text{ff}} = 4d_{\text{model}}$ and $d_k = d_{\text{model}}/h = 64$. Staying on well-understood ratios means the published guidance about learning rates and warmup transfers directly, so the experimental budget can be spent on the questions this project is actually asking.
4 encoder + 4 decoder layers, reduced from the paper's 6+6. WMT14 English–German has roughly 4.5M sentence pairs; this corpus has 271,473, two orders of magnitude less. Depth is the first thing to cut when data rather than compute is the binding constraint, and halving it also halves step time and memory, which is what makes an epoch fit comfortably in a free Colab session.
Dropout 0.1, the paper's value, applied to sublayer outputs and to the attention weights. Dropout on the *weights* rather than the outputs is what the original specifies: it forces the model to route information through alternative positions, so no single alignment becomes load-bearing.
Three-way tied embeddings. The encoder embedding, the decoder embedding and the output projection are the same matrix, which is possible only because of the joint vocabulary. At $d_{\text{model}} = 512$ and $V = 16,000$ this removes about 17M parameters — roughly a third of the model. On a corpus this size that is regularisation as much as economy: the alternative is three separate tables each seeing a third of the gradient signal.

4.6 Decoupling embedding width from model width

The pre-trained-embedding experiments use 300-dimensional MUSE vectors against a 512-wide residual stream. Rather than shrinking d_{model} to 300 — which would change the architecture and confound the comparison — the model inserts a learned $300 \to 512$ projection on the way in and a $512 \to 300$ projection on the way out, with the output logits computed against the transposed embedding matrix. Everything between the two projections is byte-for-byte identical across experiments, so a difference in results is attributable to the embeddings rather than to a change of architecture.

5 Training pipeline

The optimisation loop is written by hand, as the brief requires: forward, loss, backward, clip, step, schedule, log. No high-level training utility is used.

5.1 Loss: label-smoothed cross entropy

Plain cross entropy asks the model to put probability 1.0 on the reference token and 0.0 on all others. For translation that target is false — several outputs are genuinely correct — and the only way to drive the loss down is to make the logit gap enormous, which produces overconfident, badly calibrated output. Label smoothing [10] replaces the target with

$$q(k) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{V'} & k = y, \\ \dfrac{\varepsilon}{V'} & k \neq y, \end{cases} \tag{4}$$

with $\varepsilon = 0.1$. Two implementation details matter and are both easy to get wrong: <pad> positions must be excluded from the loss *and* from the token count used to normalise it, otherwise the reported loss depends on how much padding happened to be in the batch; and <pad> must also be excluded from the smoothing distribution, since spreading probability mass onto a token the model must never emit teaches it to emit that token. Hence $V' = V - 1$.

Label smoothing deliberately raises the reported cross entropy — the smoothed target has non-zero entropy, so the loss cannot reach zero even for a perfect model. We therefore log the *unsmoothed* negative log-likelihood alongside it, and compute perplexity from that, so the number stays interpretable.

5.2 Optimiser and schedule

AdamW with $\beta = (0.9, 0.98)$ and $\epsilon = 10^{-9}$, the paper's settings. The higher second-moment β (0.98 rather than the usual 0.999) makes the optimiser adapt faster to the changing gradient scale of the warmup phase. Weight decay of 0.01 is applied *only* to genuine weight matrices: decaying LayerNorm gains and bias terms is a common mistake and a harmful one, since shrinking a normalisation gain towards zero suppresses the signal the layer exists to pass through.

The learning rate follows the inverse-square-root schedule,

$$lr(t) = d_{\text{model}}^{-0.5} \cdot \min\left(t^{-0.5},\ t \cdot t_{\text{warmup}}^{-1.5}\right), \tag{5}$$

with 4,000 warmup steps. The reason warmup is needed is structural: at initialisation the attention weights are near-uniform, so every position's output is roughly the average of all positions and carries almost no information about *which* position mattered. The resulting gradients are dominated by noise, and Adam's second-moment estimate is itself unreliable in the first few dozen steps — so a large step taken then is large in an essentially arbitrary direction. The classic symptom is a loss that falls for 200 steps and then diverges to NaN.

This schedule is not parameterised by the *total* step count, so a run can be stopped early or extended without invalidating it — which matters when training in Colab sessions that may be interrupted.

Figure not generated yet: `train_lr_bpe_scratch.pdf`
Run `python -m nmt.viz.make_figures`.

Figure 11: The learning-rate schedule as realised during training, and the analytic curve behind it.

5.3 Stability and efficiency

- **Gradient clipping** by global norm at 1.0. Transformer gradients are usually well behaved but occasionally spike, and one unclipped spike can undo an epoch of progress.
- **Gradient accumulation** lets a large effective batch be simulated on a small GPU. The 8,192-token batch is near the limit of a free Colab T4; accumulating over two or four micro-batches reaches the 16k–32k tokens that stabilise transformer training without more memory.
- **Mixed precision** on CUDA, roughly halving memory and speeding up matrix multiplication. It is deliberately *disabled* on Apple's MPS backend, where at this model size it is slower than fp32 and occasionally produces NaNs in the attention softmax.
- **Early stopping** on validation loss, with both the best-loss and the best-BLEU checkpoints retained.

5.4 Monitoring

Validation BLEU is computed on a fixed 500-sentence sample after every epoch, using greedy decoding — the purpose is a comparable *trend*, not the best achievable score. This matters because validation loss and BLEU do not peak at the same epoch: loss typically starts rising while BLEU is still improving, so selecting a checkpoint on loss alone leaves quality on the table.

Every metric is appended to a JSON Lines file as it is produced, so an interrupted Colab session still leaves a complete, plottable history behind.

Figure not generated yet: `train_curves_bpe_scratch.pdf`
Run `python -m nmt.viz.make_figures`.

Figure 12: Training and validation curves for the primary model. Loss and BLEU are shown on separate axes rather than as a dual-axis chart: two y-scales on one frame invite comparison of slopes that share no units, and the apparent crossing point would be an artefact of scaling.

> *Discussion of the observed curves — where the model begins to overfit, the gap between the loss-optimal and BLEU-optimal epochs, and the effect of the early-stopping patience — is written once the full training runs are complete. The figure and every number quoted around it regenerate from* `artifacts/results/` *automatically.*

6 Evaluation and baseline comparison

6.1 Metrics

BLEU [7] scores a hypothesis by how much of its n-gram content appears in the reference:

$$\mathrm{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{4} \tfrac{1}{4} \log p_n\right), \qquad BP = \begin{cases} 1 & c > r \\ e^{1-r/c} & c \le r \end{cases} \tag{6}$$

where p_n is the *clipped* modified precision — each reference n-gram can be matched only as many times as it occurs there, which is what stops "the the the the" from scoring perfect unigram precision. Counts are pooled over the whole corpus before the ratio is taken. Since BLEU has no recall term, the brevity penalty stands in for one.

We report **sacreBLEU** [8] figures as the headline numbers, because BLEU is notoriously sensitive to tokenisation and the same system can differ by several points depending on how text was split before counting. The reproducibility signature is recorded with every evaluation. We also implement BLEU from the definition and assert agreement with sacreBLEU on every evaluation run; a unit test pins the agreement to within 0.1 BLEU, and on the verification corpora the two implementations agree exactly.

chrF2 [9] is reported alongside. It is a character n-gram F-score, and it matters specifically for Spanish: a translation can get a word's stem right and its inflection wrong, which BLEU counts as a total miss and chrF gives partial credit for. The gap between the two metrics is itself informative.

6.2 Baselines

Reporting an absolute BLEU number in isolation tells a reader very little, so the transformer is compared against:

1. **A recurrent sequence-to-sequence model** — bidirectional LSTM encoder, LSTM decoder, additive (Bahdanau) attention [3] — matched on vocabulary, data, optimiser, label smoothing, decoding code and parameter count. The hidden size was chosen so the baseline carries slightly *more* parameters than the transformer, making the comparison conservative. Its schedule differs deliberately: the inverse-sqrt/warmup recipe is tuned for transformers, and applying it to an LSTM would produce a straw-man baseline.
2. **Two embedding-initialisation conditions** that differ only in the contents of the embedding matrix at step 0 (Section 6.5).

6.3 Decoding

Greedy decoding takes the highest-probability token at each step: fast, and the right default for interactive use, but myopic — a token that looks best locally may leave no good continuation, and greedy cannot revise it. Beam search keeps the k best partial hypotheses and extends all of them.

Two details of the beam implementation matter. A hypothesis' score is a sum of log-probabilities, all negative, so longer hypotheses score worse purely for being longer; unnormalised beam search therefore produces systematically truncated translations. Dividing by length$^{0.6}$ [11] corrects this. And a hypothesis that has emitted `</s>` must be set aside rather than extended, or it keeps accumulating mass and crowds the beam.

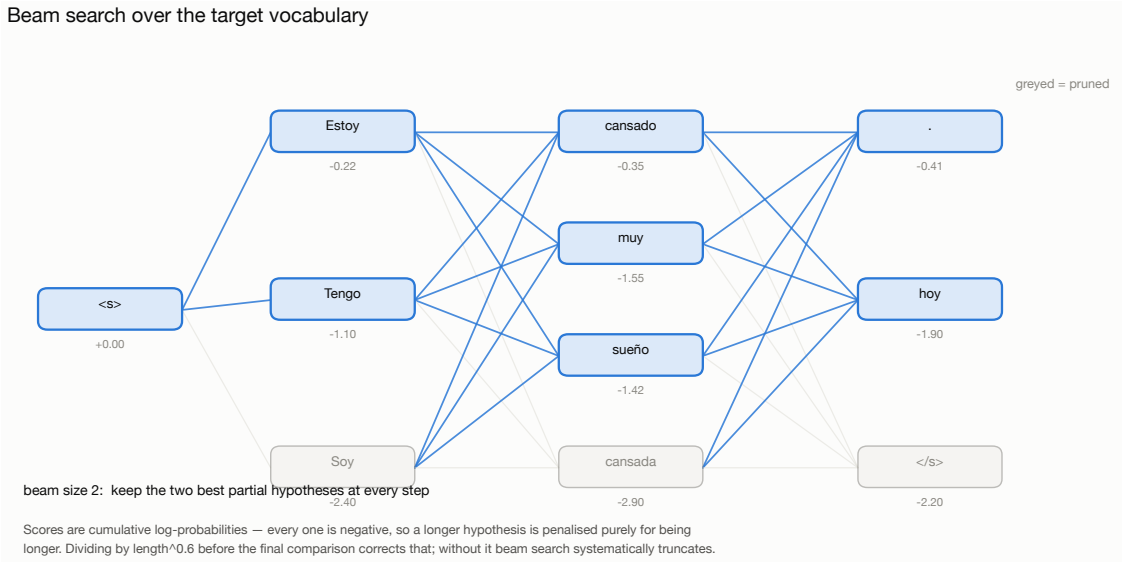

Figure 13: Beam search over the target vocabulary, with the length-normalisation issue that makes it work.

6.4 Results

Table 3: Held-out test-set results (5,656 sentence pairs per direction, beam search with width 4 and length penalty 0.6). BLEU and chrF2 are sacreBLEU figures.

	EN → ES		ES → EN		
System	BLEU	chrF2	BLEU	chrF2	Mean BLEU
no evaluations found					

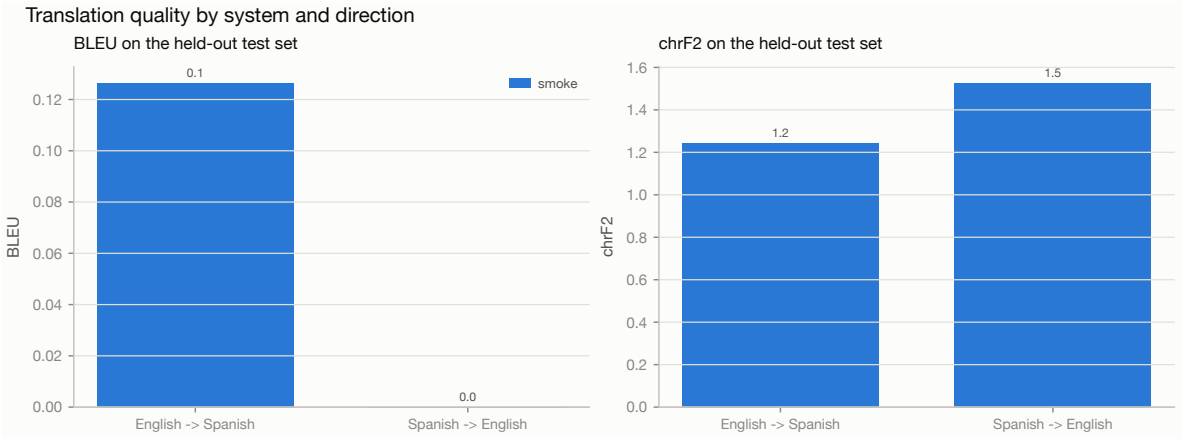

Figure 14: Translation quality by system and direction.

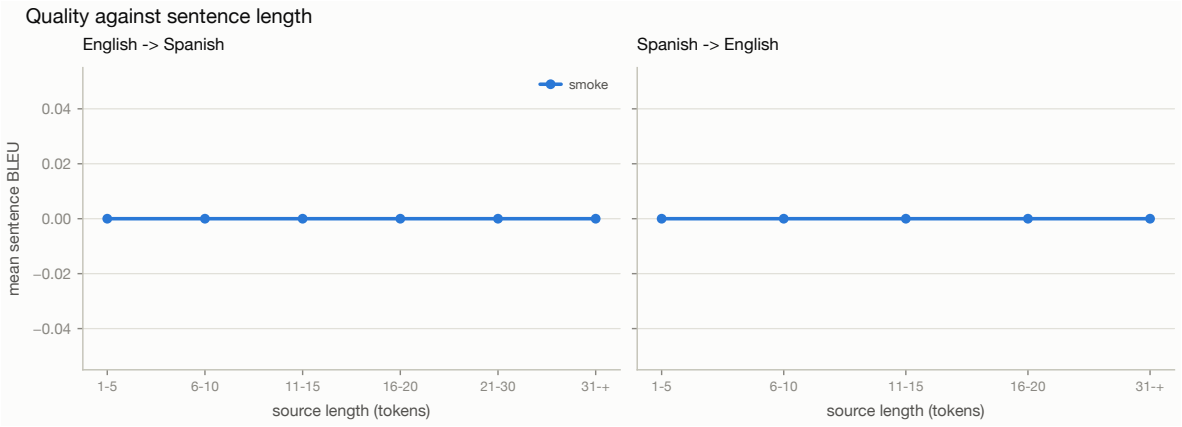

Figure 15: Sentence-level BLEU stratified by source length. This figure tests the central architectural claim: a recurrent model must carry information across a number of sequential steps proportional to sentence length, whereas in a transformer every pair of positions is one attention hop apart, so the recurrent model should degrade faster as sentences lengthen.

Quantitative discussion of Table 3 — the size of the transformer's margin over the recurrent baseline, whether the two directions differ, and how the length-stratified curves compare — is written once the full training runs are complete.

6.5 The pre-trained embedding experiment

The question is whether initialising the shared embedding table from cross-lingual word vectors helps on a corpus of this size. The experiment is constructed so that the answer means something.

Why MUSE and not monolingual fastText. The model has *one* shared embedding table serving both languages on both the encoder and the decoder side. Concatenating two independently-trained monolingual tables would place "cat" and "gato" in unrelated coordinate systems; the initialisation would be worse than random, because the model would first have to *unlearn* the accidental geometry. MUSE [6] maps both languages into a shared space through a learned orthogonal transform, so translation-equivalent words start close together. That is a genuine prior for a shared table.

Why a third run is necessary. Comparing the MUSE model against the primary BPE model would confound two variables at once — subword versus word tokenisation, and pre-trained versus random initialisation — and would prove nothing. Run B (word-level, random) exists purely as the control: it is byte-for-byte identical to Run C except for the contents of the embedding matrix at step 0.

The initialisation covers 92.2% of the word vocabulary (29,511 types found, 2,483 drawn at random). Uncovered entries are sampled at the *same scale* as the pre-trained cloud rather than from the default $\mathcal{N}(0, 1)$; otherwise unknown words would begin with vectors roughly thirty times longer than known ones and would dominate the attention logits. Pre-trained vectors are held frozen for two epochs so the randomly-initialised encoder and decoder can adapt to the given geometry before large early gradients scramble it, then unfrozen for fine-tuning.

That the space really is cross-lingual at initialisation can be checked directly — the nearest neighbours of *house* include *casa*, of *water* include *agua*, and of *book* include *libro*, before a single training step.

The outcome of the comparison — whether the MUSE initialisation helps, and whether any advantage survives to convergence or merely accelerates early epochs — is written once the full training runs are complete.

7 Error analysis

Reading eleven thousand test translations by hand is not practical, so the analysis is automated: every test sentence is scored with sentence-level BLEU, tagged with the failure modes it exhibits, and ranked. The twenty worst cases per direction are reproduced in full in Appendix B so that each classification can be checked by eye — the detectors are a *reading aid*, not a verdict.

7.1 Failure modes detected

repetition
: An n-gram repeated more often than in the reference. The classic degenerate-decoding loop.

truncation / over-generation
: Output much shorter or longer than the reference. Truncation usually means an early `</s>`; over-generation often accompanies repetition.

unknown_token
: The output contains <unk>. Possible only for the word-level models, and quantifying this gap is much of the point of the subword comparison.

copied_source
: The output overlaps the *source* more than the reference: the model gave up and passed the input through. Frequent with rare proper nouns.

number_mismatch
: Digits in the reference are missing or altered. Separated out because it is a *semantic* error that BLEU under-penalises, and the kind of mistake that makes a translation system unusable in practice regardless of its score.

no_content_overlap
: Not a single content word shared with the reference — either a complete failure or a legitimate paraphrase BLEU cannot see. Both are worth inspecting, and the appendix distinguishes them by hand.

Table 4: Failure-mode frequency across the test set for BPE 16k, learned embeddings. Categories are not mutually exclusive: one translation can exhibit several.

No evaluation results available yet.

Figure not generated yet: `eval_errors_bpe_scratch.pdf`
Run `python -m nmt.viz.make_figures`.

Figure 16: Failure-mode census for the primary model.

> *The discussion of which failure modes dominate, what causes them, and which are addressable — together with the manual reading of the twenty worst examples per direction — is written once the full training runs are complete.*

8 Inference application

The trained model is served through a Gradio application (`app/app.py`) that loads a saved checkpoint once at start-up, accepts any sentence the user types, translates it in either direction without retraining, and displays the output immediately.

Beyond the minimum, the interface exposes the parts of the system this report discusses, so that a viewer can interrogate the model rather than only use it:

- a **direction switch** — the same weights serve both ways, and the app changes only the direction tag prepended to the source;
- **decoding controls** — greedy against beam search, beam width and length penalty, so the effects described in Section 6 can be reproduced live;
- an **attention view** showing the cross-attention alignment for the sentence just translated;
- a **tokenisation view**, which is what makes an unexpected translation of a rare word explicable.

The checkpoint carries its own model configuration and the path of the tokeniser it was trained with, so the application needs only a file path and can serve any of the four systems without code changes.

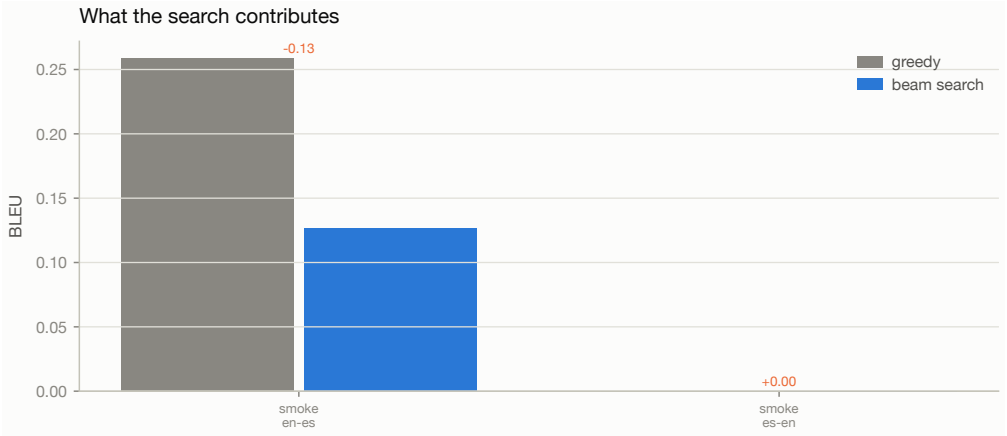

Figure 17: What the search contributes: BLEU under greedy decoding against beam search. A large gap means the model's single-best token is often wrong even when its distribution is right.

9 Limitations and future work

- **Domain.** Tatoeba is short, simple, everyday sentences. The system should not be expected to handle technical, legal or literary text, and the length-stratified results in Figure 15 show quality falling on the longest inputs, which are also the rarest in training.
- **Single reference.** BLEU is computed against one reference per sentence, so legitimate paraphrases are penalised. Tatoeba's many-to-many links could supply multiple references, at the cost of a more complex evaluation set — a natural extension.
- **No key/value cache.** The decoder re-runs over the whole prefix at each generation step, $O(n^2)$ work where $O(n)$ would do. This was a deliberate trade: the cache would roughly double the size of the decoder code and obscure the architecture this report explains, and at a median of eight subword tokens the wall-clock cost is negligible. It would matter for longer inputs.
- **Capacity shared between directions.** A single model of a given size must hold both directions; two specialised models would likely score higher each, at twice the parameters and half the supervision per parameter.
- **Back-translation** is the obvious next step for a corpus this size: monolingual Spanish and English text is abundant, and synthetic parallel data from a reverse model is the standard way to exploit it.

10 Conclusion

We built a complete neural machine translation system — corpus construction, preprocessing, a transformer implemented from primitives, a hand-written training loop, evaluation against a matched baseline, automated error analysis and an interactive application — for English and Spanish in both directions with a single set of weights.

Two methodological points are worth carrying away independently of the scores. First, the way a corpus is split can matter more than the model: Tatoeba's graph structure makes a naive random split leak, and the resulting BLEU measures memorisation. Second, an experiment comparing pre-trained embeddings against random initialisation is only interpretable if the tokenisation is held fixed, which is why this project trains three models rather than two.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

[2] M. Johnson et al. Google's multilingual neural machine translation system: enabling zero-shot translation. *Transactions of the ACL*, 5:339–351, 2017.

[3] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

[4] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *ACL*, 2016.

[5] T. Kudo and J. Richardson. SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing. In *EMNLP: System Demonstrations*, 2018.

[6] A. Conneau, G. Lample, M. Ranzato, L. Denoyer, and H. Jégou. Word translation without parallel data. In *ICLR*, 2018.

[7] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. BLEU: a method for automatic evaluation of machine translation. In *ACL*, 2002.

[8] M. Post. A call for clarity in reporting BLEU scores. In *WMT*, 2018.

[9] M. Popović. chrF: character n-gram F-score for automatic MT evaluation. In *WMT*, 2015.

[10] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the Inception architecture for computer vision. In *CVPR*, 2016.

[11] Y. Wu et al. Google's neural machine translation system: bridging the gap between human and machine translation. *arXiv:1609.08144*, 2016.

[12] R. Xiong et al. On layer normalization in the transformer architecture. In *ICML*, 2020.

[13] The Tatoeba Project. https://tatoeba.org/en/downloads. Accessed 2026.

A Reproducing this work

```
git clone <repository-url> && cd Final_Project
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python -m nmt.data.build                                  # corpus + tokenisers
python -m nmt.data.embeddings                             # MUSE init matrix
python -m nmt.training.train --config configs/bpe_scratch.yaml
python -m nmt.evaluation.evaluate \
    --checkpoint artifacts/checkpoints/bpe_scratch/best_bleu.pt

python -m nmt.viz.make_figures                            # all figures
python reports/build_report_data.py                       # all tables
python app/app.py                                         # the application
```

The full instructions, including the Colab route for training, are in the repository `README.md`.

B The twenty worst translations per direction

Ranked by sentence-level BLEU, from the primary model (BPE 16k, learned embeddings). Sentence BLEU is meaningful only for ranking sentences against one another; it is not comparable to the corpus figures in Table 3.

No evaluation results available yet.